ML4NLP1 · Week 07 · Self-study

The Computation Graph of an LSTM

There is no in-person tutorial this week. Instead, please work through this sheet on your own; it should take about 50 minutes. Computation graphs and RNNs are both typical exam topics, so it is well worth the time.

> **Guiding question:** why can an LSTM carry information across many more time steps than a simple RNN? You will draw the complete computation graph of one LSTM step by hand and then see, with actual numbers, where the gradient vanishes and where it does not.

Interactive version with a live chart: open in your browser. If the link does not work, download W07_lstm_computation_graph.html from the course repository and open it locally.

What you need

- Pen and paper (or any drawing tool you like) and a calculator.
- Simon's autodiff visualizer, which you know from the lecture. Every Open ↗ link below opens it with an expression and values already filled in. The visualizer computes with concrete numbers, so each part gives you values for the inputs and the weights. These values are invented for the exercise; in a real network, the weights would be learned. The *Visual Diagram* tab shows the graph with the forward value and the gradient (the overlined value) in each node, and the *Step-by-Step Walkthrough* tab lists every local derivative. You can change the values and run it again.

Please draw each graph first and only then open the link: comparing your own drawing with the result is where most of the learning happens.

To keep the graphs readable, we use a single hidden unit, so all vectors and matrices become single numbers. The structure of the graph stays the same as with vectors. The formulas have no bias terms.

Part 1: One step of a simple RNN 10 min

$$h_j = \tanh(x_j\, w^x + h_{j-1}\, w^h)$$

Values: $x_j = 1$, $h_{j-1} = 0.1$, $w^x = 0.5$, $w^h = 0.5$.

1. Draw the computation graph. The leaf nodes are the inputs x_j and h_{j-1} and the weights w^x and w^h.
2. **Forward pass:** write the values into the leaves and compute the value of every other node, up to h_j.
3. **Backward pass:** start with the gradient 1 at h_j and compute the gradient of h_j with respect to every other node, down to the four leaves. Hint: the derivative of $\tanh(a)$ is $1 - \tanh^2(a)$. The visualizer writes it as `cosh(a)**(-2)`, that is $1 \,/\, \cosh^2(a)$, which is the same function.
4. ❓ **On its way from h_j back to h_{j-1}, the gradient is multiplied by two local derivatives. Which ones?**
5. Check your graph and your numbers in the visualizer, where the variables are typed as `x`, `h`, `wx` and `wh`.

   One RNN step ↗

Part 2: The full LSTM step by hand 20 min

These are the LSTM formulas in Goldberg's notation:

$$c_j = f \odot c_{j-1} + i \odot z \qquad h_j = o \odot \tanh(c_j)$$

$$i = \sigma(x_j\, W^{xi} + h_{j-1}\, W^{hi}) \qquad f = \sigma(x_j\, W^{xf} + h_{j-1}\, W^{hf})$$

$$o = \sigma(x_j\, W^{xo} + h_{j-1}\, W^{ho}) \qquad z = \tanh(x_j\, W^{xz} + h_{j-1}\, W^{hz})$$

Values: $x_j = 1$, $h_{j-1} = 0.5$, $c_{j-1} = 0.3$, and the weights below (typed as `wxi`, `whi` and so on in the visualizer).

$$W^{xi} = 0.4 \qquad W^{hi} = -0.3$$

$$W^{xf} = 1 \qquad W^{hf} = 0.6$$

$$W^{xo} = 0.3 \qquad W^{ho} = 0.5$$

$$W^{xz} = 0.8 \qquad W^{hz} = 0.5$$

Part 2: The full LSTM step by hand continued

Draw the complete computation graph (forward pass only, no gradients) of one step, and label the operator nodes with the names from the formulas (f, i, o, z, c_j, h_j).

Some help:

- The leaf nodes are the inputs x_j, h_{j-1} and c_{j-1} and the eight weights.
- x_j and h_{j-1} each feed into all four gates.
- Start with one gate, for example o, and check it in the visualizer. The other three gates follow the same pattern.

  Output gate ↗

Then compare your drawing with the complete step. Full LSTM step ↗ The visualizer numbers the intermediate nodes `n1`, `n2`, ..., and each node shows how it is computed from the previous ones, for example `n3 = n1 + n2`. So first find out which of them are f, i, o, z and c_j. The easiest way is to start from the weights, whose names contain the gate: following the edges from `wxf` and `whf` upwards, the σ node you reach is f. To check, compute f by hand and compare it with the value of that node.

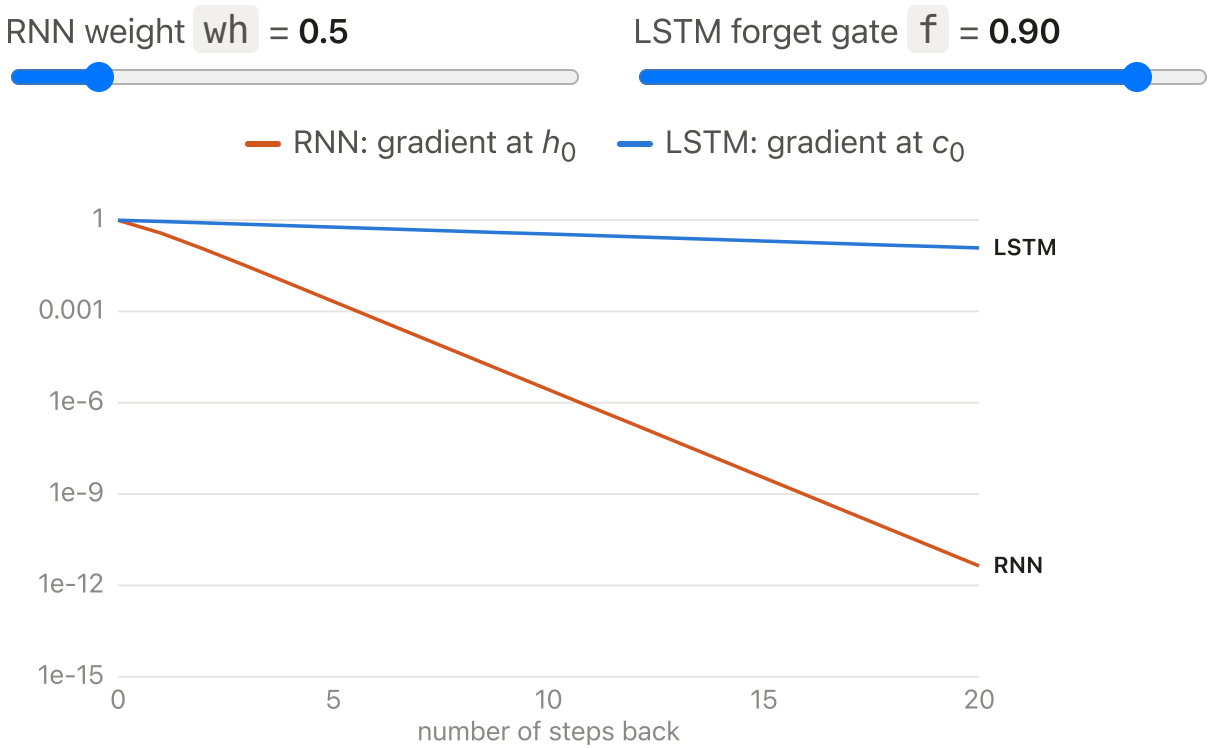
Part 3: The cell state 10 min

> **Tip:** the two diagrams in this part show both the forward and the backward pass, which makes them hard to read in the visualizer itself. Download each of them as a PDF under *Export*, below the diagram, and zoom in there.

1. In your drawing, mark the path from c_{j-1} to c_j. ❓ **Which operations lie on it?** Compare them with the path from h_{j-1} to h_j in Part 1.
2. Open the cell state update, which uses the values from Part 2. ❓ **The gradient at `c` equals the value of one of the nodes. Which one, and why?** Cell state update ↗
3. Now the cell state runs for five steps. To focus on the cell state, the gates are fixed numbers here: $f = 0.9$, $i = 0.5$, $z = 0.5$, and the cell starts at $c_0 = 0.1$. Read off the gradient at `c0`. Then set `f` to 1 and to 0.5. Five cell state steps ↗
4. ❓ **What does an LSTM have to learn in order to keep a piece of information for many steps? And what happens when f is close to 0?**

Part 4: RNN and LSTM over twenty steps 10 min

Finally, let us compare the two over many steps. The chart shows the gradient that reaches h_0 when the RNN from Part 1 runs for twenty steps, and the gradient that reaches c_0 along the cell state from Part 3. The RNN uses the values from Part 1 and receives the same input $x = 1$ at every step. Note that the vertical axis is logarithmic: every grid line is a factor of 1,000.



After 5 steps: RNN **2.14e-3**, LSTM **0.59**. After 20 steps: RNN **4.37e-12**, LSTM **0.12**.

The sliders work in the interactive version of this sheet (see the first slide).

Part 4: RNN and LSTM over twenty steps continued

The two sliders are independent and do not need to have the same value, since they are different kinds of numbers: `wh` is a weight of the RNN, while `f` is the value of the LSTM's forget gate, which always lies between 0 and 1. What you compare is how much of the gradient survives each step back. In the RNN, it is multiplied by w^h and by the derivative of tanh; along the cell state, it is multiplied only by f.

1. With the default values, compare the two gradients after one, five and twenty steps. If you hover over the chart, it shows the exact numbers.
2. Move the `wh` slider and try to make the RNN gradient after twenty steps as large as possible. ❓ **Why does it remain tiny, however you set `wh`?**

   Hint

   Look at the two factors separately. What happens to their product when w^h is small? And what happens to tanh and its derivative when w^h is large?
3. Now move the `f` slider. ❓ **How close to 1 does f have to be so that, after twenty steps, the gradient is still at least 0.1? What happens when f is small?**
4. ❓ **What does this mean for a model that has to connect a word with another word twenty positions earlier?**

In a real LSTM, the gates themselves depend on h_{j-1}, so there are further paths through the gates, and the gradient along those can vanish just as in the RNN. The direct path through the cell state, however, remains.

Check your answers

Answers: Part 1

Forward pass: $x_j\, w^x = 0.5$ and $h_{j-1}\, w^h = 0.05$, so the sum is 0.55 and $h_j = \tanh(0.55) \approx 0.50$.

Backward pass: the gradient at the input of tanh is $1 - {h_j}^2 = 1 - 0.50^2 \approx 0.75$. The addition passes it on unchanged to both products, and each multiplication passes it on multiplied by the other factor. The gradients at the leaves are therefore $0.75 \cdot w^x \approx 0.37$ at x_j, $0.75 \cdot x_j = 0.75$ at w^x, $0.75 \cdot w^h \approx 0.37$ at h_{j-1} and $0.75 \cdot h_{j-1} \approx 0.07$ at w^h.

On the way to h_{j-1}, the gradient is thus multiplied by the derivative of tanh (0.75) and by the weight $w^h = 0.5$, which gives about 0.37.

The visualizer shows the same 0.75 as `cosh(n3)**(-2)`: since $\cosh^2 - \sinh^2 = 1$, dividing by $\cosh^2$ gives $1 - \tanh^2 = 1 \,/\, \cosh^2$.

Answers: Part 2

The forget gate is $f = \sigma(1 \cdot 1 + 0.5 \cdot 0.6) = \sigma(1.3) \approx 0.79$. In the full step, f = 0.79, i = 0.56, z = 0.78, c_j = 0.68, o = 0.63 and h_j = 0.37. In the visualizer, these are the nodes `n8` (f), `n13` (i), `n17` (z), `n19` (c_j), `n4` (o) and `out` (h_j). Your drawing should contain four gate blocks of the same shape (two multiplications by weights, an addition and a σ or tanh), all fed by the same x_j and h_{j-1}. On top of them, c_j is the sum of two elementwise products, and h_j is the product of o and $\tanh(c_j)$.

Last year's solution slide (the last page) shows the same graph for vectors; you can ignore the state s_j at the end.

Answers: Part 3

On the path from c_{j-1} to c_j there is only one elementwise multiplication by f and one addition: no tanh and no weight. Therefore, the gradient at `c` equals the value of f (0.79): in $c_j = f \cdot c_{j-1} + i \cdot z$, only the first term depends on c_{j-1}.

Over five steps, the gradient becomes f^5: 0.59 for $f = 0.9$. With $f = 1$, the gradient passes through unchanged, and with $f = 0.5$ it drops to 0.03. An LSTM therefore has to learn to keep f close to 1 for information it wants to remember, while the input gate i decides what is written into the cell. A forget gate close to 0 deliberately erases the memory, which can be useful, for example, at the start of a new sentence.

Answers: Part 4

After one step, the RNN gradient is 0.37, as in Part 1. After five steps, it is only `2.14e-3`, and after twenty steps `4.37e-12`, whereas the gradient along the cell state is still 0.59 and 0.12 (that is, 0.9^5 and 0.9^{20}).

At every step, the RNN gradient is multiplied by w^h and by the derivative of tanh. This derivative is at most 1. With small weights, the product is small anyway; with large weights, the tanh saturates: the hidden state quickly gets close to 1 (about 0.99 for `wh=2`), so the derivative of tanh is close to 0. Either way, the gradient shrinks with every step: the largest value after twenty steps that you can reach is about `1.57e-11`, at `wh=0.7`.

For a simple RNN, this means that a word twenty positions earlier has almost no influence on the weight updates, and the model can hardly learn such long-range dependencies. If w^h times the derivative of tanh stays above 1, the gradient can explode instead.

Along the cell state, f has to be at least 0.9, since $0.9^{20} \approx 0.12$ but $0.89^{20} \approx 0.097$. With a small f, the cell state loses the gradient just as quickly as the RNN: for $f = 0.27$, only `4.24e-12` is left after twenty steps. The difference is that the LSTM learns its gates, so it can keep information when it is needed and erase it when it is not.

Further reading

Chris Olah's blog post Understanding LSTM Networks explains the same ideas with very clear diagrams.

If you have any questions, please ask them in the OLAT forum.